

C++ UNIX/Linux File Indexer

Systems Architecture, Reliability, and Performance Report

Project report

17 August 2026

Abstract

This report documents a C++17 file indexer evolved into a UNIX/Linux systems project without changing its word-count, Top-K, or version-difference query semantics. The implementation combines POSIX file I/O, optional memory-mapped input, a bounded producer-consumer pipeline, worker-local indexing, optional process isolation with pipe IPC, and crash-safe persistent versions. The design prioritizes correct token handling at chunk boundaries, bounded in-flight work, and recoverable persistence. Benchmarks on a warmed 64 MiB synthetic log show that `mmap` is modestly faster than `read()` in the single-worker case and that worker-local indexing scales to 4.80x at eight workers.

Contents

1	Project Goal and Scope	3
2	Architecture	4
3	Input and Token Correctness	4
3.1	POSIX buffered I/O	4
3.2	Memory-mapped alternative	4
3.3	Boundary-safe parallel work	5
4	Concurrency and Synchronization	5
4.1	Bounded producer-consumer queue	5
4.2	Index ownership model	5
5	Process Isolation and IPC	5
6	Crash-Safe Persistent Versions	6
7	CLI and Reproducibility	6
8	Benchmark Methodology	6
9	Benchmark Results	7
9.1	Reader mode comparison	7
9.2	Worker scaling with <code>mmap</code>	7
10	Interpretation and Bottlenecks	9

1 Project Goal and Scope

The original project indexed large text/log files using a fixed-size buffer and answered three queries:

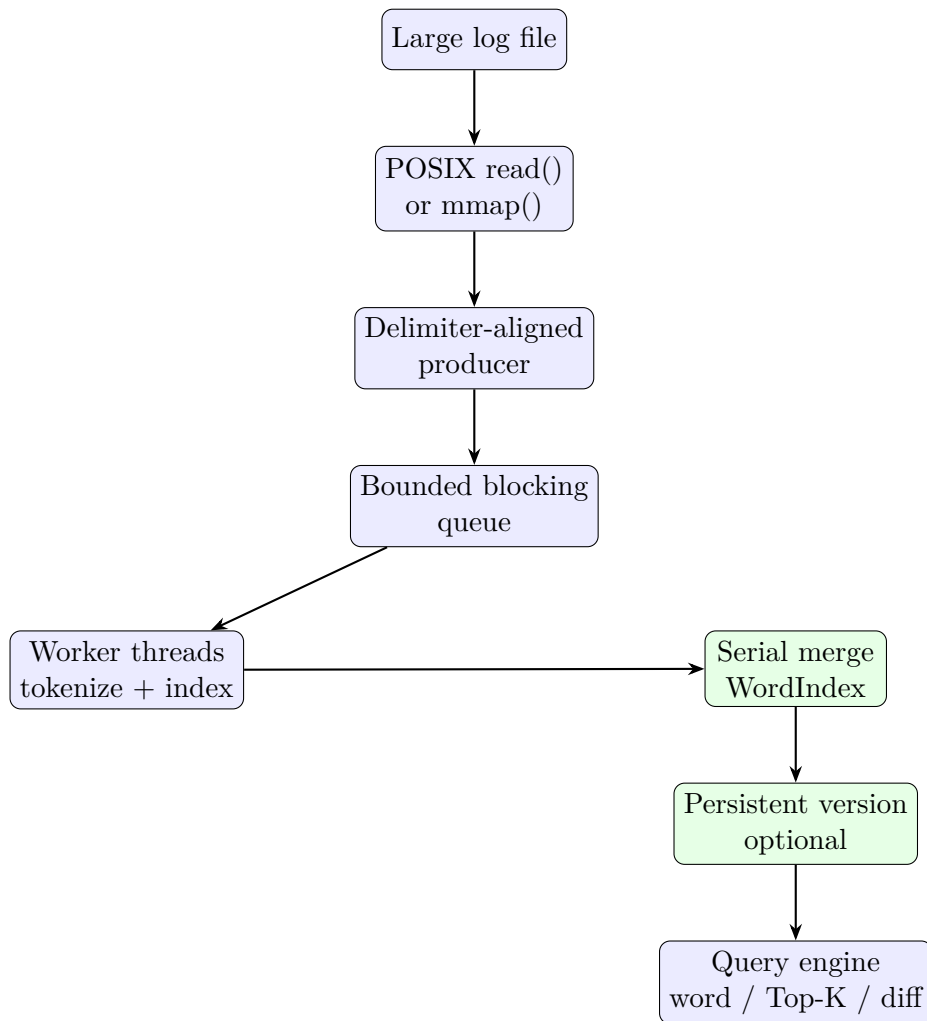
- case-insensitive word-frequency lookup;
- Top-K word-frequency queries; and
- a word-frequency difference between two named versions.

The systems upgrade retained those observable semantics while adding operating system concepts where they improve the implementation:

- POSIX `open()`, `read()`, `write()`, `close()`, `fsync()`, and `rename()`;
- optional `mmap()` input with `munmap()` cleanup;
- a mutex/condition-variable bounded queue;
- parsing and local index construction on multiple `std::threads`;
- `fork()`, `exec()`, `pipe()`, and `waitpid()` for optional process-isolated indexing; and
- durable, version-addressed index files.

No external runtime dependency was added. The repository provides a `Makefile`, test executables, and shell benchmark scripts.

2 Architecture



The default path runs in one process. With `-process-indexer`, the parent remains the query process and starts a child per version build. The child executes the same binary in an internal export mode, and the completed `WordIndex` is transferred over a pipe using a fixed-width binary protocol. This optional mode improves fault isolation and establishes a service-like boundary; it is not intended to maximize throughput because a complete index must be serialized.

3 Input and Token Correctness

3.1 POSIX buffered I/O

`BufferedReader` owns a file descriptor and a 256 KiB–1 MiB buffer. It retries `read()` when interrupted by `EINTR`, reports errors through exceptions, and closes the descriptor through `RAII` cleanup. This replaces stream I/O while preserving the original chunk API used by the tokenizer.

3.2 Memory-mapped alternative

The `mmap` reader validates that its input is a regular file, maps it privately with `PROT_READ | MAP_PRIVATE`, and presents the mapping in the same configured chunk sizes. It therefore changes

the transport mechanism, not tokenization or query semantics. Empty files are handled without calling `mmap()` with a zero length.

3.3 Boundary-safe parallel work

Tokenization treats contiguous alphanumeric bytes as one lower-cased word. Independent workers cannot safely tokenize arbitrary byte chunks because a word can span a chunk boundary. The producer solves this by retaining an unfinished suffix and emitting work only through the most recent non-alphanumeric delimiter. Every queued chunk is therefore independently tokenizable. The final suffix is flushed at end of file.

This scheme keeps queued work bounded in count. Extremely long individual words can still make the producer's retained suffix grow, which is required to preserve exact word semantics and is a documented edge case rather than a hidden truncation.

4 Concurrency and Synchronization

4.1 Bounded producer-consumer queue

`BoundedBlockingQueue<T>` is a multi-producer, multi-consumer queue with a fixed capacity of eight chunks in the indexing pipeline. A mutex guards the deque and closed state. Two condition variables express the exact blocking conditions:

- `notFull`: producers wait while the queue is full;
- `notEmpty`: consumers wait while the queue is empty; and
- `close()`: wakes both sides, rejects future pushes, and lets consumers drain already queued work.

Using condition variables is preferable to adding semaphores here because the queue needs both a capacity predicate and a drain-after-close predicate.

4.2 Index ownership model

Each worker owns one private `WordIndex`; no worker mutates another worker's hash map. After all threads join, the caller merges the local maps serially. This removes a shared hash-map mutex from the hot token-update path. The queue and the worker exception pointer are the only shared mutable states, and both are synchronized. Thread joins establish the happens-before boundary before the merge.

5 Process Isolation and IPC

`-process-indexer` creates a unidirectional pipe, forks, redirects the child's standard output to the pipe, then execs the same executable with an internal index-export flag. The parent reads and validates:

1. a protocol magic value;
2. the number of entries;
3. each fixed-width word length, word bytes, and signed frequency.

Partial reads and writes, including `EINTR`, are handled explicitly. The parent closes the pipe and uses `waitpid()` to reject signaled or nonzero-exit children. This avoids treating an incomplete IPC transfer as a valid index.

6 Crash-Safe Persistent Versions

Passing `-index-dir PATH` saves every newly built version. The persistence transaction is:

1. create a unique temporary file in the destination directory;
2. serialize the index and `fsync()` the temporary file;
3. close it and atomically replace the target using `rename()`;
4. on Linux, `fsync()` the containing directory to make the rename durable; and
5. remove the temporary file if a pre-rename step fails.

The target name is derived from a hex encoding of the logical version name, so version names cannot traverse out of the configured directory. Loading is explicit through `-load-index`; the program never substitutes a persisted version for a requested source file without the user's instruction.

7 CLI and Reproducibility

```
make
```

```
make test
```

```
./build/file-indexer --query word --reader mmap --workers 4 \  
  --file data/log.txt --version v1 --buffer 1024 --word error
```

```
./build/file-indexer --query word --file data/log.txt --version v1 \  
  --buffer 1024 --index-dir indexes --word error
```

```
./build/file-indexer --query word --load-index --index-dir indexes \  
  --version v1 --word error
```

`make test` builds and executes four focused tests:

- bounded-queue backpressure, closure, and multi-producer/multi-consumer behavior;
- a word deliberately crossing a 256 KiB input boundary with both `read` and `mmap` modes;
- index serialization round-trip and invalid-frequency rejection; and
- persistent save, load, and atomic replacement.

8 Benchmark Methodology

All reported benchmarks use a warmed, 64 MiB synthetic text log containing repeating alphanumeric words, a 1 MiB chunk size, and three iterations. The workload is deliberately simple and should not be treated as a universal production result. It is useful for comparing implementation modes under the same conditions.

Latency is the application's reported complete index-build time. Throughput is computed as input bytes divided by average latency. The benchmark invokes a missing-word query, ensuring that every run builds the full index. Peak-memory measurement is supplied by `benchmark_memory.sh`:

it uses `/usr/bin/time -l` on macOS and `/usr/bin/time -v` on Linux. Restricted environments may deny access to memory counters; the script reports that condition rather than inventing a value.

9 Benchmark Results

9.1 Reader mode comparison

Table 1: Single-worker reader benchmark, warmed 64 MiB input. Lower latency and higher throughput are better.

Reader	Average latency (s)	Throughput (MiB/s)
<code>read()</code>	0.24233	264.10
<code>mmap()</code>	0.22633	282.77

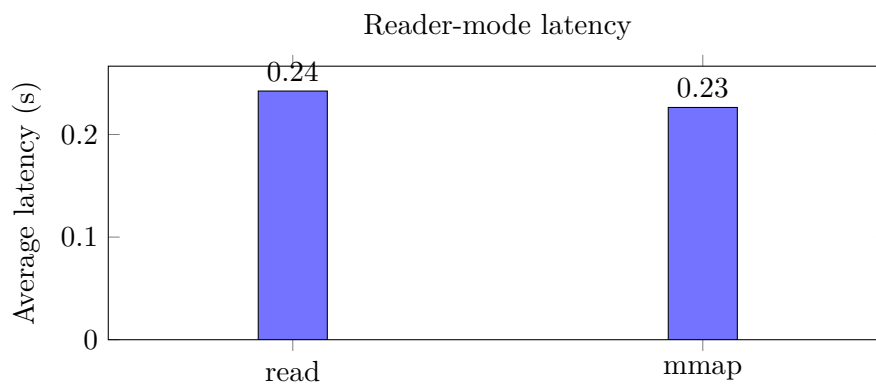


Figure 1: The `mmap` mode was about 6.6% faster in this warmed, single-worker workload.

9.2 Worker scaling with `mmap`

Table 2: `mmap` scaling benchmark, warmed 64 MiB input.

Workers	Average latency (s)	Throughput (MiB/s)	Speedup
1	0.22512	284.29	1.00x
2	0.11830	541.00	1.90x
4	0.07030	910.38	3.20x
8	0.04689	1364.90	4.80x

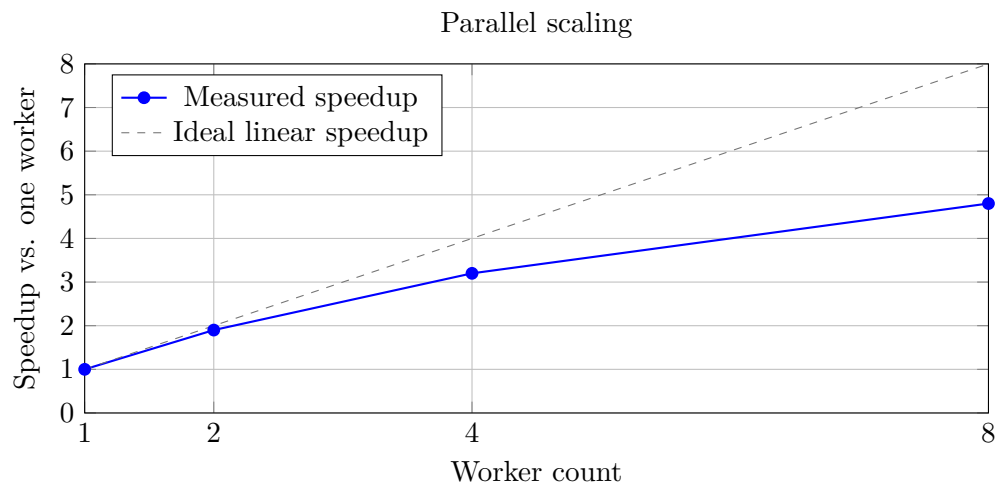


Figure 2: Scaling is strong but sublinear. At eight workers, the serial producer, memory subsystem, and final merge increasingly limit throughput.

10 Interpretation and Bottlenecks

The measurements show three useful conclusions:

1. **Reader choice matters, but not alone.** The warmed `mmap` advantage was small because tokenization and hash-map updates dominate much of the end-to-end work.
2. **Worker-local maps scale well.** Removing a shared-map mutex lets workers process different delimiter-aligned chunks with little contention at low and moderate worker counts.
3. **Amdahl’s law remains visible.** The producer is single-threaded, and the final map merge is serial. These portions, together with memory bandwidth, explain the gap between 4.80x measured and 8x ideal speedup.

Expected bottlenecks on more diverse workloads are:

- producer delimiter scanning and queue coordination at high worker counts;
- allocation, lowercasing, and hashing for high-cardinality vocabularies;
- page faults and storage bandwidth on cold data;
- final local-map merge cost for many unique words;
- complete-index pipe serialization in process-isolated mode; and
- synchronous file and directory durability work for persistent versions.

For a production evaluation, record CPU model, available memory, compiler, operating system, storage medium, reader mode, chunk size, worker count, file size, token distribution, and whether the page cache was cold or warm. Cold-I/O tests and warmed CPU-scaling tests answer different questions and should be reported separately.

11 Conclusion

The project is now a complete systems-oriented file indexer. Its core query semantics remain simple, while its implementation demonstrates practical UNIX techniques: descriptor-based I/O, memory mapping, bounded synchronization, multithreading, process boundaries, binary pipe IPC, and crash-safe persistence. The implementation favors explicit tradeoffs over resume-driven features: process mode is optional because it costs serialization time, and worker-local indexes avoid a shared-map lock because that lock would degrade the hot path. The included tests, Makefile, and benchmark scripts make those choices reproducible and measurable.